dMIR Peephole Rewrites for Consensus-Critical EVM JITs with First-Class Carry Operators

Anonymous Author(s)

Anonymous Institution
anonymous@example.com

Abstract. Blockchain JIT peephole rewrites must preserve transaction semantics bit-for-bit; any divergence between consensus-path VMs can cause chain forks. In LLVM IR, extracting carry and overflow from aggregate intrinsic returns fragments carry-dependent rewrites across SSA nodes, blocking single Alive2 queries and forcing implementers to drop such rules, ship them unverified, or hand-prove them one by one. dMIR (DTVM's middle IR) represents carry and overflow as first-class bit-vector operators, so each dMIR rewrite can be expressed as a single Z3 equivalence query within a two-tier system. An offline Z3 admission and coverage pipeline checks the 70 production dMIR rewrites under fixed-width bit-vector semantics, including carry-chain rewrites enabled by first-class ADC/SBB operators. A separate x86 CgIR layer adds 13 cleanup rewrites that are validated by matched differential testing rather than SMT. The full 83-rule system achieves a 7.24% geometric-mean speedup on 27 evmone-bench workloads, with 5884 differential Cancun tests reporting byte-identical final state and matching gas accounting across the suite (correctness bounded by suite coverage). This design addresses a verification bottleneck for peephole rewrites in consensus-critical engines; more broadly, domain-specific IR with first-class bit-vector operators provides a reusable path for SMT-checkable rule design in such engines.

Keywords: Blockchain information systems · EVM JIT · dMIR · Peephole optimization · SMT equivalence checking · Carry chain · Translation validation

1 Introduction

The execution layer of a blockchain information system is a replicated consensus state machine: every node on the consensus path must produce bit-identical results for the same transaction. The EVM is a widely deployed smart-contract execution environment; when a client enables a JIT, the compiler becomes part of the consensus-critical path. Peephole rewrites in the JIT act directly at machine-code generation, so their correctness bears directly on the determinism of on-chain execution; a wrong equivalence judgment, deployed on only some clients, may trigger a chain fork.

Existing formal verification of EVM peephole rewrites mostly targets the bytecode block-level abstraction. The machine-level patterns emitted by a JIT during code generation—especially the multi-word carry chains that arise when the 256-bit word length is lowered onto 64-bit hardware—sit at a different abstraction layer from bytecode rules and are awkward to validate as single stock LLVM/Alive2 peephole queries. The root cause is how IRs model bit-level side-products. In LLVM IR, basic arithmetic is defined only on N-bit integers; the carry bit must be retrieved through an intrinsic that returns a multi-return tuple of {result, carry}. Rules depending on this side-product can be expressed only through indirect structures in Alive [13] and similar tools; the bottleneck is the IR's expressiveness, not the SMT solver [14]. For a consensus-critical JIT, an unchecked rule class leaves the engineer with three choices—drop it (performance), ship it without verification (safety), or hand-prove it (scalability).

Our approach: extend the IR with first-class bit-vector operators so that SMT-checkable peephole rules cover a larger surface, without modifying the solver or relaxing the differential-test threshold. In dMIR, the bit-level patterns that previously required indirect multi-return encoding become first-class operators, so the corresponding rules fall within Z3 bit-vector equivalence. The SMT scope is dMIR arithmetic/flag semantics; gas, memory, storage, and x86 equivalence remain outside it.

Contributions.

1. **A dMIR extension for SMT-checkable carry-chain peepholes.** First-class carry operators make the four cases in Table 1 single Z3 bit-vector queries rather than indirect LLVM/Alive2 tuple encodings. The two-tier rule set has **70** dMIR rules (SMT-checked for arithmetic/flags) and **13** x86 rules (differentially tested; no separate SMT check).
2. **An enumerable, Z3-admitted search space covering the production dMIR set.** A capacity run evaluates **853** candidates (**98.7%** algebraic, **21.8%** carry admitted by Z3); a separate coverage run admits **1966** rules whose canonical LHSs cover all **70/70** hand-designed production rules. This is post-hoc reachability, not blind recall (§4.1).
3. **End-to-end performance and correctness.** Across the **27** EVM benchmarks of evmone-bench, the geometric-mean speedup is **+7.24%** (95% confidence interval [+2.59%, +12.11%], which excludes zero), with a peak of **+25.79%**; **5884** differential tests report byte-identical final state and identical gas accounting in a same-engine DTVM self-A/B comparison.

Relative to the EVM bytecode block-level formal rule set of Albert et al. [2] at CAV 2023, the rule set in this paper operates at the JIT intermediate representation and machine-code layers that the bytecode abstraction cannot reach (quantitative comparison in §5).

2 Background and Motivation

2.1 Hardware Cost and Carry-Chain Shape of U256 Arithmetic

The native word length of the EVM is 256 bits, whereas general-purpose registers on current commodity hardware are 64 bits wide. An EVM-level U256 addition expands, after JIT code generation, into four machine-level additions: one ordinary 64-bit `add` followed by three carry-propagating `adc` instructions that form a *serial carry chain*. Subtraction is analogous, using one `sub` and three `sbb` instructions. The listing below shows the 4-limb code at the dMIR and x86-64 levels for a typical U256 addition:

dMIR (4-limb addition)	**x86-64 output**
`r0 = add(a0, b0)`	`add rax, rcx`
`r1 = ADC(a1, b1, cout0)`	`adc rbx, rdx`
`r2 = ADC(a2, b2, cout1)`	`adc rsi, rdi`
`r3 = ADC(a3, b3, cout2)`	`adc r8, r9`

Among all instructions emitted by the JIT, instructions directly tied to U256 multi-word arithmetic account for about **2.15%** (weighted mean across the U256-heavy subset of the 27 benchmarks in §4—the U256-heavy cluster in Figure 1). The share is small, but the carry chain has a special property: every `adc`/`sbb` on the chain depends on the carry output of the previous instruction, forming a strict *serial data dependency* that instruction-level parallelism cannot hide. Despite its small share, the chain is still worth optimizing: on U256-heavy workloads, rule hit rates reach 25–99.9% (§4.2). Section 4.2 reports per-family hit rates: on U256-heavy benchmarks the U256 rule hit rate ranges from 25% to 99.9% with a very uneven distribution; on non-U256-heavy benchmarks (sha1 family, `jump_around`) it is well below that range, and gains come from x86-layer mechanical cleanup.

2.2 Carry-Chain Expressiveness Limits in General-Purpose IRs

In LLVM IR, basic arithmetic (`add`, `sub`, `mul`) is defined only on N-bit integers `iN`; to obtain the carry bit, the programmer must call `@llvm.uadd.with.overflow.iN`, which returns a `{iN, i1}` multi-return tuple. The carry is *not a native type* but hidden in the second field of a struct. Following the Alive/Alive2 [13,12] peephole verification framework, Table 1 lists four carry-chain rules that Alive2 cannot directly verify (type mismatch on the multi-return tuple, or timeout on the lifted encoding); the limitation stems from IR carry modeling, independent of solver capacity.

The four rules' Alive2 command lines, failure messages, and tool versions (Alive2 v21.0, Z3 4.8.12) are archived in the artifact (§6).

The bottleneck is not the SMT solver [14]: the six U256 rules without multi-return carry pass Alive2 in ∼60 ms each.

In LLVM IR, expressing one carry-propagating addition takes five lines around `uadd.with.overflow`; in dMIR, a single `ADC` suffices. Native carry-bit support brings all four rules into the scope of automatic equivalence checking.

Table 1. Four typical carry-chain rules that Alive2 (based on LLVM IR) cannot directly verify for equivalence because of the multi-return tuple encoding. The first three are single `ADC`/`SBB` rules where the right-hand side drops the carry output; the fourth is a 4-limb carry-chain merge rule.

Rule	Rewrite
`adc_zero_carry`	$\texttt{ADC}(x, y, 0) \to \texttt{add}(x, y)$
`sbb_zero_borrow`	$\texttt{SBB}(x, y, 0) \to \texttt{sub}(x, y)$
`sbb_self_zero`	$\texttt{SBB}(x, x, 0) \to 0$
4-limb ADC chain	$4 \times$ `uadd.w.overflow` $\to$ `add.i256`

3 SMT-Checked dMIR Peephole Rewrites

Offline scripts run Z3 admission and coverage checks over dMIR rule metadata; the JIT runs a compiled two-tier implementation—dMIR for algebraic rewrites and x86 CgIR (code-generation IR) for carry-chain cleanup. The dMIR bit-vector model is in §3.1; enumeration and admission are in §3.2.

3.1 dMIR Bit-Vector Semantics and Carry-Chain Operators

The core data type of dMIR is a parametric-width bit-vector BV_n with BV_1 carrying the carry in/out (c_{in}, c_{out}). The carry-propagating addition has signature $\texttt{ADC} : \mathrm{BV}_n \times \mathrm{BV}_n \times \mathrm{BV}_1 \to \mathrm{BV}_n \times \mathrm{BV}_1$; `SBB` has analogous borrow-propagation semantics. In Z3 [14], carry propagation zero-extends both 256-bit operands and the 1-bit carry to 257 bits, sums once, and slices: the low 256 bits are the result and bit 256 is carry out. This embeds carry propagation entirely in bit-vector arithmetic with no multi-return tuple or auxiliary predicate. The model covers dMIR arithmetic and flags; gas, memory aliasing, storage, and x86 equivalence remain outside it. The same semantics covers ordinary operators (`add`, `sub`, `mul`, `and`, `or`, `xor`, shifts); the carry-chain operator is the operator that general-purpose IRs lack.

Trusted base. The SMT check assumes the dMIR bit-vector model agrees with the JIT runtime; the 5884 differential tests of §4.3 show no divergence (random-bytecode fuzzing is future work, §5.3).

3.2 Candidate Enumeration and SMT Admission

Admission check. For a candidate $LHS \to RHS$, we submit to Z3 the proposition $\exists\, \boldsymbol{v}.\ LHS(\boldsymbol{v}) \neq RHS(\boldsymbol{v})$. *Unsat* grants admission under fixed-width modular bit-vector semantics; *sat* or timeout rejects (sat returns a counterexample). Equivalence assumes nothing beyond modular arithmetic—no signed interpretation and no undefined-overflow assumptions. All 70 production dMIR rules pass the offline Z3 admission check, and CI re-runs this check on every change to the rule file.

Two enumeration runs. We report two separate enumeration experiments with disjoint goals. The *capacity run* fixes a single grammar—a depth-2 algebraic enumerator plus a small set of hand-written carry templates—and measures the Z3 admission rate; this characterizes solver throughput and the typical false-positive boundary. The *coverage run* extends the same algebraic enumerator with additional left-hand-side operator tops and a constant-pool widening, plus a pattern-template instantiation channel reusing the template miner configuration; this measures whether the tuned offline search space can reach the 70 hand-designed dMIR production rules, not blind recall on an unseen rule set. Coverage results appear in §4.

The capacity run uses two complementary paths: *(i) depth-2 algebraic enumeration*—all left/right combinations of depth ≤ 2 over the dMIR arithmetic and Boolean operators, covering common algebraic identities; *(ii) carry-template enumeration*—multi-word carry-chain candidates constructed by hand around the carry-chain operators.

Throughput and admission rate. The capacity run admits 765/775 algebraic candidates (98.7%, 10 timeouts) and 17/78 carry templates (21.8%, 61 semantic rejections): carry-template admission is limited by 61 semantic rejections, mostly width-mismatch edge cases (full breakdown in the artifact), while the algebraic side has only 10 timeouts and zero semantic rejections. Full data: Table 3 (§4). These candidates measure enumeration capacity; only the curated production rules are loaded into the JIT at runtime.

Production rule coverage (methodology). The coverage run asks a different question: does the enumeration search space contain the 70 hand-designed production dMIR rules? We compare canonicalized left-hand sides. The *alpha-normalized* canonical form renames variables to a fixed sequence $x_1, x_2, \ldots$ in traversal order, so that two rules differing only in operand naming map to the same representative; commutative pairs (e.g. $\texttt{or}(x, y)$ and $\texttt{or}(y, x)$) likewise collapse to one representative. A production rule counts as *covered* when at least one enumerated rule has the same canonical left-hand side; the *false-miss rate* is the fraction of production rules with no such match. The match is on canonical left-hand sides only—the enumerated right-hand side is verified equivalent to the left-hand side by Z3 but is not required to be syntactically identical to the production rule's right-hand side. Numerical results appear in §4.

Counterexamples. Two rejected carry templates illustrate why SMT admission is essential on the carry chain. $\texttt{SBB}(x, x, c_f) \to 0$ fails at $x{=}0, c_f{=}1$ (LHS is $\texttt{0xFF}\ldots\texttt{F}$; correct form $\texttt{sub}(0, c_f)$); $\texttt{SBB}(x, 0, c_f) \to \texttt{sub}(0, c_f)$ fails at $x{=}1, c_f{=}0$ (correct form $\texttt{sub}(x, c_f)$). Both reflect modular-carry traps: ignored carry input and confused operand polarity. The artifact contains ∼2400 lines of Python (enumerator, template miner, canonical comparator, Z3 admission checks), the rule-set JSON, coverage output, and unit tests for canonicalization/dedup.

Table 2. Classification of the two-tier peephole rewrite rules by evidence type. The dMIR rules are checked for equivalence by Z3 on the bit-vector semantic model; the x86 CgIR rules are covered by matched differential tests, not by SMT.

Layer	Evidence	Category	Count
dMIR	Z3 equivalence check	Boolean normalization	42
		Algebraic identity	15
		Identity elimination	10
		Constant folding	2
		Dead-code elimination	1
		Subtotal	**70**
x86 CgIR	Differential tests	Redundant compare/test cleanup	8
		Branch-fallthrough cleanup	2
		No-op elimination	2
		test-setcc merge	1
		Subtotal	**13**
Total			**83**

Two-tier rule set. The dMIR layer handles machine-independent algebraic simplification (all 70 dMIR rules pass Z3); the x86 CgIR layer handles hardware-specific cleanup on registers, flags, and branches (13 syntactic rewrites with **no separate SMT script**, covered by the matched 5884-test suites of §4.3 and Table 4). A formal x86 SMT model is future work (§5.3).

Table 2 groups the rules by logical class. Boolean normalization and algebraic identities together cover most dMIR rules—these are precisely the patterns the canonical-form enumerator (§3.2) is designed to cover. Identity-elimination and shift-zero patterns admit Z3-discharged proofs in milliseconds, while the carry-chain class is the one that motivates the operator extension of §3.1.

Selection asymmetry. The 70 dMIR production rules are selected manually from recurring profile hotspots and lowering patterns, then submitted one by one to Z3; this is hotspot-guided curation, not a claim of systematic optimality. The enumeration pipeline is a measurement channel: it does not drive production rule maintenance, and the 1966 admitted candidates are not automatically promoted to production. The x86-layer rules are covered by code review and matched test suites. The limitations from this asymmetry are listed in §5.3.

4 Evaluation

4.1 Enumeration and Admission Output vs. the General-Purpose IR

Table 3 reports the enumeration/admission output of §3.2; Alive2's verification on the same rule sample is summarized below.

Table 3. Output of the enumeration/admission pipeline: candidate counts, Z3 passes, SAT counterexamples, timeouts, and admission rate. The two enumeration paths are described in detail in §3.2.

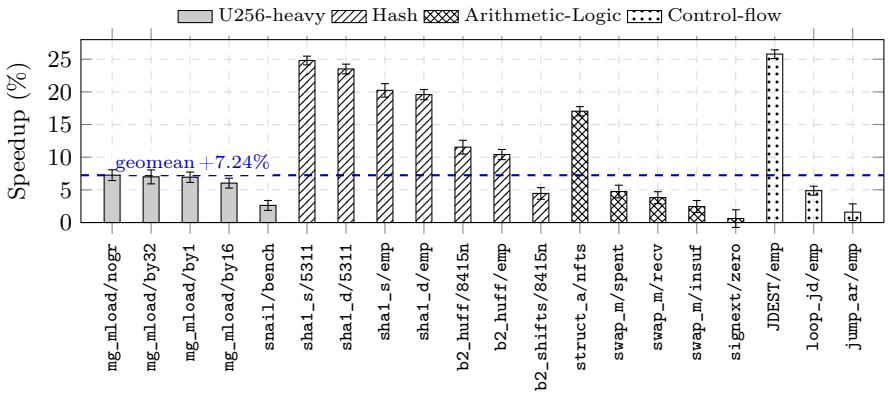
Candidate source	Input	Pass	SAT cex	Timeout	Admission rate
Algebraic enumeration	775	765	0	10	98.7%
Carry templates	78	17	61	0	21.8%

SAT counterexamples are semantic rejections; timeouts are conservatively rejected without counterexamples.

Z3 discharges the hardest 256-bit queries in ∼60 ms and rechecks all 70 dMIR rules in ∼0.5 s, so width is not the bottleneck. On a 10-rule U256 capacity-run sample, Alive2 clears the 6 without multi-return carry but reports type mismatch or timeout on the 4 `adc`/`sbb` cases. This sample is separate from the four production-style cases in Table 1; both expose the same tuple cause.

Production rule coverage. Using the canonical-LHS methodology of §3.2, the coverage run produces **1966** Z3-admitted rewrite rules: **1856** from the extended algebraic enumerator and **110** from pattern-template instantiation. The pattern-template path starts from 2964 pattern-template candidates, which Z3 verification and deduplication against the enumerator output and the existing rule file reduce to the final 110. On the 70 hand-designed production rules, the run matches **69/69** unique canonical patterns—one commutative-twin pair, $\texttt{or}(x, y)$ versus $\texttt{or}(y, x)$, shares a single canonical representative that both production names match—equivalently **70/70** production rule names; the false-miss rate against the production set is therefore **0/70**.

Scope of the coverage claim. The claim has two boundaries. First, the result measures coverage of the *existing* production rule set; the extended enumerator's left-hand-side operator tops and constant pool were broadened in light of an earlier 0/70 baseline that exposed mechanical gaps (missing tops such as `not`/`select`/`shl`/`ushr`/`sshr` as left-hand-side roots, missing small constants, and structural-equality dedup of forms such as $\texttt{and}(x, x)$). Reaching 70/70 therefore demonstrates that the production rules lie inside an enumerable and Z3-discharged search space; it does not demonstrate blind recall of an unseen target. Second, coverage is defined on canonical left-hand sides; the enumerated right-hand side is verified equivalent to its own left-hand side by Z3 but is not required to be syntactically identical to the production rule's right-hand side. Together with the admission rate above, this result shows the tuned search space is broad enough to contain every hand-designed rule, while 1966 of its members pass Z3 verification; it does not claim that the non-production members are useful runtime rewrites.



Labels: mg=`memory_grow`, b2=`blake2b`, sha1_s=`sha1_shifts`, sha1_d=`sha1_divs`, struct_a=`struct_array`, swap_m=`swap_method`, signext=`signextend`, JDEST=`JUMPDEST_n0`, loop_jd=`loop_jumpdest`, jump_ar=`jump_around`, snail=`snailtracer`.

Fig. 1. Positive-speedup benchmark configurations from enabling the peephole rewrite rule set over the upstream-main baseline (error bars are 95% bootstrap confidence intervals, B=10 000). The horizontal line at zero marks the no-change baseline; the dashed line at **+7.24%** marks the 27-benchmark geometric-mean speedup. From left to right, the benchmarks are grouped into four workload clusters: U256-heavy, Hash, Arithmetic-Logic, and Control-flow.

4.2 End-to-End Performance

Setup. Intel Core Ultra 7 265K/Ubuntu 22.04; evmone-bench, baseline (upstream `main`) vs. rules-enabled side, 20 runs (30 if CV $\geq$ 3%), bootstrap CIs (B=10 000), pinned CPU, turbo off.

Across all 27 benchmarks, the full 83-rule set improves the geometric mean by **+7.24%** (95% CI [+2.59%, +12.11%]); the positive-speedup cases are shown in Figure 1. Peak speedup is **+25.79%** on `JUMPDEST_n0/empty`. The `sha1` family gains +19.59%–+24.81%, and `blake2b_huff` gains +10.41%–+11.54%.
Hit rate and speedup are decoupled: `mg_mload` matches nearly every U256 instruction yet gains under 8%, while sha1 hits only 7.5% of opcodes yet gains over 20% from x86 branch and compare-merge cleanup. The two layers complement each other: multi-word rules concentrate gains on U256-heavy contracts, x86 cleanup yields broader gains on ordinary code.

Compilation overhead. Across 776 JIT unit-test modules, median per-module compile time is 0.45 ms (p95 0.87 ms) covering the full loading–analysis–codegen pipeline; this is an upper bound on the whole pipeline rather than the incremental rewrite-pass cost (per-pass attribution is future work, §5.3, item 4).

4.3 Correctness and Rule Coverage

Both sides agree item-by-item across the four differential categories of Table 4 (5884/5884 total). Independently, `evmone-statetest` on the Cancun fork

Table 4. Results on four differential test categories for the rules-enabled side (83 added rules) and the baseline side (upstream main): passes/total. Matching numbers in both columns indicate that the enabled rule set introduces no correctness regression.

Test dimension	Enabled	Baseline	Suite source
Unit (JIT)	223/223	223/223	EVM spec subset
Unit (interpreter)	215/215	215/215	Run list
statetest (JIT)	2723/2723	2723/2723	Cancun 101 suites
statetest (interpreter)	2723/2723	2723/2723	Cancun 101 suites
Total	**5884/5884**	**5884/5884**	—

reaches the same matrix under multipass JIT and interpreter modes.[1] The suite covers Cancun state tests and EVM unit tests, but we do not claim opcode- or gas-branch coverage completeness.

CI re-runs the same admission check on every change to the rule file (timing in §4, distinct from the per-module JIT compile time of §4.2). Comparison with the CAV 2023 rule set of Albert et al. [2] appears in §5.

5 Related Work and Discussion

5.1 Peephole Optimization Frameworks

We position our work by abstraction layer, verification method, and carry-chain coverage. Classical synthesize-then-verify systems include Denali [10], Bansal–Aiken [5], STOKE [20], Stratified Synthesis [8], and egg [21]; SMT-based peephole checking is exemplified by Alive [13], Alive2 [12], Souper [18], and PEEK [17]; dataflow filtering [15] accelerates search; recent LLM-driven peephole synthesis on LLVM IR [22] is a generative complement to deductive enumeration. We follow the verify-after-enumeration paradigm but move the checked IR from LLVM IR to dMIR, addressing the multi-return carry and overflow encoding limit that motivates this paper.

LeanMLIR [6] formalizes IR-parametric peephole verification in Lean 4, whereas this paper reports an instantiated EVM JIT rule set. Hydra [16] generalizes bit-width-independent rules but, like Alive2, builds on LLVM IR and inherits the multi-return tuple limit on the four carry-chain examples in Table 1; closing that gap via an auxiliary lifted encoding is orthogonal to our IR-extension contribution. Schett and Nagele [19] enumerate SMT-checked EVM *bytecode* rules; we instead target the JIT code-generation path with dMIR matching and x86 cleanup.

[1] Cancun is selected via `evmone-statetest -k fork_Cancun`; Prague tests are excluded because DTVM does not yet support that fork.

5.2 Verification Work on the Consensus-Critical Path

Albert et al. [2] give a Coq proof of AOT EVM bytecode-block rules. Our scope sits at lower abstraction layers: JIT IR and machine-code, SMT admission at synthesis time, and multi-word carry chains. Seven rules overlap: 7/83 in our set and 7/14 in their `forves` set. This supports a complementary-scope reading rather than a superset claim: the remaining 76 cover dMIR algebraic and Boolean simplifications (including ADC/SBB carry-chain rewrites that motivate §3.1) plus 13 x86 cleanup rewrites, none of which are reachable from the bytecode-block abstraction.

Runtime divergence detection—Hydra Framework [7], multi-client detectors, EVM fuzzing, consensus test networks—catches faults after deployment; our workflow validates rules before deployment. Bytecode-side EVM optimization [3,9,1,4,11] is orthogonal to our JIT IR/code-generation focus.

5.3 Limitations and Future Work

- **(1) Trusted base and x86 coverage.** The SMT check assumes the dMIR model agrees with JIT semantics; the 13 x86 rules rely on paired differential runs. Formal x86 modeling is future work.
- **(2) Workload, baseline, and modeling scope.** Speedups come from a same-engine upstream-`main` comparison on 27 evmone-bench benchmarks, not evmone/revm; only arithmetic and flags are SMT-checked, leaving gas, memory aliasing, and storage unmodeled.
- **(3) Dead-carry inertness at U256 heads.** A U256 add/sub head has a runtime third operand rather than matchable zero, so the rule is inert there but still effective inside U256 multiplication lowering.
- **(4) Compilation overhead.** Median 0.45 ms and p95 0.87 ms per module are rules-enabled-side full-pipeline numbers; EVMC exposes no per-stage timing hook, so we cannot attribute the cost to individual passes, and baseline-side timing requires a separate instrumented build.
- **(5) Corpus mining provenance.** An initial dump (v1) emitted shared-subtree compression placeholders that the rule-parser did not consume; v1 produced 237 distinct shapes at Jensen–Shannon divergence 0.28 against held-out test corpora. We re-implemented the dump pass to fully inline subtrees (v2); v2 produces 1,415 distinct shapes at JS = 0.15 with 0.4% test out-of-vocabulary mass. The shift was larger than our pre-registered band predicted; investigation attributes it to per-MFunction local back-references in v1 fragmenting shape identity across functions and concentrating mass in a single OOV bucket. The v1 dataset is archived for reproducibility; a v1↔v2 top-20 shape ablation appears in the supplementary material. One test benchmark (`blake2b_huff`) drops 8.5% of records under our 64 KiB per-record subtree budget due to deep U256 sub-expression chains; analysis confirms exponential DAG fan-out (raising the budget to 1 MiB only reduces the drop to 8.6%) and the retained 91.5% covers `blake2b_huff`'s shape distribution.

6 Conclusion

This paper builds a two-tier EVM JIT peephole system: 70 dMIR rules pass Z3 checks over arithmetic and flags, while 13 x86 cleanup rules are covered by differential runs. First-class carry and overflow operators let a domain IR cover cases that bytecode-level identities alone cannot. Future work covers two fronts. On verification, a formal x86 model would replace the differential-test fallback for the x86 layer, and a complete Alive2 classification would tighten the comparison with the LLVM IR baseline. On systems, cross-engine baselines (e.g. evmone, revm), dataflow pruning of the enumeration search, and composition with bytecode-level rule sets are natural extensions.

Data and Code Availability

Rule sets, offline Z3 admission checks, evaluation scripts, and data are available in an anonymous repository upon request to the program chairs and will be de-anonymized upon acceptance. CI runs the production dMIR validation tests on every change to the rule file.

References

1. Aguiar, M.A., Albert, E., Genaim, S., Gordillo, P., Hernández-Cerezo, A., Kirchner, D., Rubio, A.: Neural-guided superoptimization in Ethereum. Information and Software Technology (2025). https://doi.org/10.1016/j.infsof.2025.107800
2. Albert, E., Genaim, S., Kirchner, D., Martin-Martin, E.: Formally verified EVM block-optimizations. In: Computer Aided Verification (CAV). LNCS, vol. 13966, pp. 176–189 (2023). https://doi.org/10.1007/978-3-031-37709-9_9
3. Albert, E., Gordillo, P., Rubio, A., Schett, M.A.: Synthesis of super-optimized smart contracts using Max-SMT. In: Computer Aided Verification (CAV). LNCS (2020). https://doi.org/10.1007/978-3-030-53288-8_10
4. Avigad, J., Goldberg, L., Levit, D., Seginer, Y., Titelman, A.: A proof-producing compiler for blockchain applications. Journal of Automated Reasoning (2025). https://doi.org/10.1007/s10817-025-09723-y
5. Bansal, S., Aiken, A.: Automatic generation of peephole superoptimizers. In: Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (2006). https://doi.org/10.1145/1168857.1168906
6. Bhat, S., Keizer, A., Hughes, C., Goens, A., Grosser, T.: Verifying peephole rewriting in SSA compiler IRs. In: 15th International Conference on Interactive Theorem Proving (ITP). LIPIcs, vol. 309, pp. 9:1–9:20 (2024). https://doi.org/10.4230/LIPIcs.ITP.2024.9
7. Breidenbach, L., Daian, P., Tramèr, F., Juels, A.: Hydra: Principled bug bounties for smart contract security. In: 27th USENIX Security Symposium (USENIX Security 18) (2018)
8. Heule, S., Schkufza, E., Sharma, R., Aiken, A.: Stratified synthesis: Automatically learning the x86-64 instruction set. In: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) (2016). https://doi.org/10.1145/2908080.2908121

9. Hu, X., Zhao, D., Scholz, B.: Synthesizing efficient super-instruction sets for Ethereum virtual machine. In: Proceedings of the 16th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL) (2024). https://doi.org/10.1145/3689490.3690403
10. Joshi, R., Nelson, G., Randall, K.: Denali: A goal-directed superoptimizer. In: Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation (PLDI) (2002). https://doi.org/10.1145/512529.512566
11. Krijnen, J.O., Chakravarty, M.M., Keller, G., Swierstra, W.: Translation certification for smart contracts. Science of Computer Programming (2024). https://doi.org/10.1016/j.scico.2023.103051
12. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for LLVM. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI) (2021). https://doi.org/10.1145/3453483.3454030
13. Lopes, N.P., Menendez, D., Nagarakatte, S., Regehr, J.: Provably correct peephole optimizations with alive. In: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) (2015). https://doi.org/10.1145/2737924.2737965
14. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (2008). https://doi.org/10.1007/978-3-540-78800-3_24
15. Mukherjee, M., Kant, P., Liu, Z., Regehr, J.: Dataflow-based pruning for speeding up superoptimization. Proc. ACM Program. Lang. **4**(OOPSLA) (2020). https://doi.org/10.1145/3428245
16. Mukherjee, M., Regehr, J.: Hydra: Generalizing peephole optimizations with program synthesis. Proc. ACM Program. Lang. **8**(OOPSLA1) (2024). https://doi.org/10.1145/3649837
17. Mullen, E., Zuniga, D., Tatlock, Z., Grossman, D.: Verified peephole optimizations for CompCert. In: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) (2016). https://doi.org/10.1145/2908080.2908109
18. Sasnauskas, R., Chen, Y., Collingbourne, P., Ketema, J., Taneja, J., Regehr, J.: Souper: A synthesizing superoptimizer (2017)
19. Schett, M.A., Nagele, J.: Populating the peephole optimizer of a smart contract compiler. In: 2nd Workshop on Formal Methods for Blockchains (FMBC 2020). OASIcs (2020). https://doi.org/10.4230/OASIcs.FMBC.2020.3
20. Schkufza, E., Sharma, R., Aiken, A.: Stochastic superoptimization. In: Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (2013). https://doi.org/10.1145/2451116.2451150
21. Willsey, M., Nandi, C., Wang, Y.R., Flatt, O., Tatlock, Z., Panchekha, P.: egg: Fast and extensible equality saturation. Proc. ACM Program. Lang. **5**(POPL) (2021). https://doi.org/10.1145/3434304
22. Xu, Z., Xu, H., Tian, Y., Zhou, X., Sun, C.: LPO: Discovering missed peephole optimizations with large language models. In: Proceedings of the 31st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (2026). https://doi.org/10.1145/3779212.3790184